# NvS

# Nachvollziehbare Schritte

Reproducible end-to-end build instructions from source data to final Power BI delivery

| | |
|---|---|
| Project | End-to-End Business Intelligence and Predictive Analytics Solution using Databricks and Power BI |
| Author | Dr. Mohammad Reza Eidi |
| Data source | Google BigQuery public dataset - TheLook E-Commerce |
| Technology | Google BigQuery -> Databricks (Bronze/Silver/Gold + ML) -> Power BI |

Submission documentation | 01 October 2026

# Contents

01 Reproduction Goal and Required Files
02 Step 1 - Acquire Source Data from BigQuery
03 Step 2 - Stage CSV Files in Databricks
04 Step 3 - Build Bronze Tables
05 Step 4 - Data Quality and Silver Layer
06 Step 5 - Gold Dimensional Model
07 Step 6 - Machine Learning Workflow
08 Step 7 - Persist ML Outputs
09 Step 8 - Connect Power BI
10 Step 9 - Build the Power BI Semantic Model
11 Step 10 - Create DAX Measures
12 Step 11 - Build the Six Report Pages
13 Step 12 - Validation and Submission
14 Expected Results Checklist

# 1. Reproduction Goal and Required Files

The purpose of NvS is to make the project understandable and rebuildable by another person. The steps below describe the implemented workflow in execution order and state the expected checks after each stage.

**Rebuild order: Source -> Databricks Volume -> 01 Bronze -> 02 Silver -> 03 Gold -> 04 ML -> Power BI Model -> Report -> Export**

| Item | Requirement |
|---|---|
| Source | Google BigQuery public dataset TheLook E-Commerce |
| Databricks | Workspace with Spark/PySpark and permission to create Delta tables |
| Staging path | /Volumes/workspace/default/bi_final_raw/ |
| Notebook archive | BI_Final_Project.dbc |
| Power BI | Power BI Desktop with Databricks connector |
| Report evidence | PROJECT.pdf (6 pages) |

# 2. Step 1 - Acquire Source Data from BigQuery

Open Google BigQuery and use the public dataset:

```
bigquery-public-data.thelook_ecommerce
```

Export the four required tables:

| BigQuery table | Required purpose |
|---|---|
| users | Customer attributes, country/city and traffic_source |
| orders | Order header, user_id, status and lifecycle timestamps |
| order_items | Transactional item rows, product_id and sale_price |
| products | Product name, brand, category, department, cost and retail_price |

Save the exports as CSV. Large tables can be split into multiple parts; the Databricks code uses wildcard patterns, so all matching files are read together. The project used file families such as users_*.csv, orders_*.csv and order_items_*.csv, plus products.csv.

# 3. Step 2 - Stage CSV Files in Databricks

Upload all CSV files to the Databricks Volume path below:

```
/Volumes/workspace/default/bi_final_raw/
```

Check the uploaded file names. In the project, some files initially contained a duplicated .csv.csv extension; the notebook includes a rename step. After staging, the folder should contain users parts, orders parts, order_items parts and products.csv.

# 4. Step 3 - Build Bronze Tables

Run notebook 01_Bronze_Load. Read each CSV family with header=true and inferSchema=true, then add an ingestion timestamp and store the data as Delta tables.

```
from pyspark.sql.functions import current_timestamp

users = (spark.read.option("header", True).option("inferSchema", True)
      .csv("/Volumes/workspace/default/bi_final_raw/users_*.csv"))

(users.withColumn("_ingested_at", current_timestamp())
 .write.format("delta").mode("overwrite")
 .saveAsTable("workspace.default.bronze_users"))
```

Use the same pattern for orders, order_items and products. Expected Bronze row counts are:

| Bronze table | Expected rows |
|---|---|
| bronze_users | 100,000 |
| bronze_orders | 125,545 |
| bronze_order_items | 182,295 |
| bronze_products | 29,120 |

**Important reproducibility correction**

The exported DBC reads df_orders in Notebook 01 and later notebooks reference workspace.default.bronze_orders, but the explicit Delta write for bronze_orders is not visible in the current Notebook 01 export. To rebuild from scratch, add or confirm this command immediately after df_orders is loaded:

```
(df_orders
 .withColumn("_ingested_at", current_timestamp())
 .write.format("delta").mode("overwrite")
 .saveAsTable("workspace.default.bronze_orders"))
```

This makes the dependency explicit and ensures Notebook 02 can run in a new workspace.

# 5. Step 4 - Data Quality and Silver Layer

Run 02_Data_Quality_Silver. Load the four Bronze tables and perform the following checks before cleaning:

- Count total rows and distinct primary keys.
- Check duplicate IDs for users, orders, order_items and products.
- Generate null-count reports for every column.
- Check referential integrity using left-anti joins: orders without users, order items without orders, and order items without products.

| Check | Expected result |
|---|---|
| Duplicate user/order/item/product IDs | 0 for all four tables |
| Orders without matching user | 0 |
| Order items without matching order | 60 rows / 43 distinct order IDs |
| Order items without matching product | 0 |

Create the Silver layer. users, orders and products pass through unchanged. For order_items, keep only rows with a valid order_id using an inner join to bronze_orders.

```
silver_order_items = (
   bronze_order_items.alias("oi")
   .join(bronze_orders.select("order_id").alias("o"), "order_id", "inner")
)
```

Write all Silver tables in Delta format with mode="overwrite" and validate the final counts:

| Silver table | Expected rows |
|---|---|
| silver_users | 100,000 |
| silver_orders | 125,545 |
| silver_order_items | 182,235 |
| silver_products | 29,120 |

# 6. Step 5 - Gold Dimensional Model

Run 03_Gold_Data_Model. Load the Silver tables and create a star-schema-oriented analytical model.

| Gold table | Construction |
|---|---|
| dim_users | Select customer ID, name, email, age, gender, state, city, country, traffic_source, created_at |
| dim_products | Select product ID, name, brand, category, department, cost, retail_price, sku, distribution_center_id |
| fact_orders | Select order_id, user_id, status and lifecycle timestamps plus num_of_item |

| | |
|---|---|
| fact_order_items | Select item identifiers, user/order/product keys, status/timestamps and sale_price |
| dim_date | Generate one row per calendar day from minimum to maximum order date |

Enrich fact_order_items by joining product cost and calculating financial measures at row level:

```
revenue = sale_price
cost    = product_cost
profit  = sale_price - product_cost
```

Build dim_date with date, year, month, month_name, quarter, day, day_name and year_month. Save all five Gold tables as Delta tables.

| Gold table | Expected rows |
|---|---|
| dim_users | 100,000 |
| dim_products | 29,120 |
| dim_date | 2,821 |
| fact_orders | 125,545 |
| fact_order_items | 182,235 |

# 7. Step 6 - Machine Learning Workflow

Run 04_ML_Second_Purchase_Prediction after Gold is complete. The notebook reads Gold tables as source data and follows a leakage-safe chronological modeling design.

## 7.1 Define the business target

1. Exclude Cancelled orders when defining purchases.
2. Order each customer's non-cancelled purchases chronologically.
3. Keep only the first purchase for modeling.
4. Set label=1 when a second non-cancelled purchase occurs within 180 days; otherwise label=0.
5. Exclude first orders that do not yet have a complete 180-day future observation window.

Expected mature cohort: 56,570 first-order customers, with approximately 11.48% positive labels.

## 7.2 Engineer safe features

Build features using only information available at or before the first order: item count, order value, average item price, order cost, gross margin, gross margin percentage, distinct products/categories/brands, average retail price, customer tenure, age, cyclical month/day/hour features, weekend flag, gender, country and traffic_source. Keep state for audit but exclude it from the model vector. Exclude identifiers and all future/target-derived fields.

## 7.3 Split chronologically

| Split | Rows | Positive rate | Purpose |
|---|---|---|---|
| Train | 39,602 | 8.70% | Fit preprocessing and model |
| Validation | 8,484 | 15.30% | Compare models and lock threshold |
| Test | 8,484 | 20.63% | One-time final evaluation |

The 70/15/15 chronological split avoids future data leaking backward in time. Fit imputers, indexers, encoders and scalers on Train only. Compute class weights from the Train labels to address imbalance.

## 7.4 Train and select models

| Model | Validation PR-AUC | Selected threshold |
|---|---|---|
| Logistic Regression | 0.231 | 0.50 |
| Random Forest | 0.225 | 0.45 |
| Gradient-Boosted Trees | 0.211 | 0.45 |

Select Logistic Regression because it has the highest Validation PR-AUC and is easier to interpret. Refit the final preprocessing/model on Train + Validation using development class weights, then evaluate once on Test with the locked 0.50 threshold.

| Final Test metric | Expected value |
|---|---|
| ROC-AUC | 0.697 |
| PR-AUC | 0.334 |
| Precision | 30.6% |
| Recall | 76.3% |
| F1 | 0.436 |
| Accuracy | 59.3% |
| Specificity | 54.9% |
| Brier Score | 0.221 |
| TN / FP / FN / TP | 3700 / 3034 / 415 / 1335 |

Validate ranking quality by score decile. Expected observed repurchase rates are about 6.0% in the lowest decile and 37.7% in the highest decile. Because weighted scores are not perfectly calibrated, present the output as a Repurchase Score rather than an exact probability.

# 8. Step 7 - Persist ML Outputs

Continue through the final save cell. It writes only new ml_* tables; existing Bronze, Silver and Gold tables are not modified.

```
default.ml_second_purchase_predictions
default.ml_second_purchase_test_predictions
default.ml_second_purchase_model_metrics
```

| ML output | Expected rows |
|---|---|
| ml_second_purchase_predictions | 12,235 |
| ml_second_purchase_test_predictions | 8,484 |
| ml_second_purchase_model_metrics | 6 |

The current scoring population contains first-time customers who have not yet made a second purchase and whose 180-day outcome window is incomplete. Segment cutoffs are approximately 0.491 (Low to Medium) and 0.577 (Medium to High).

# 9. Step 8 - Connect Power BI

Open Power BI Desktop, connect to Databricks, select the workspace/catalog/schema containing the Gold and ML output tables, and load the following seven tables using Import mode:

| Load into Power BI |
|---|
| dim_date |
| dim_products |
| dim_users |
| fact_order_items |
| fact_orders |
| ml_second_purchase_predictions |
| ml_second_purchase_model_metrics |

The test-predictions table is retained in Databricks for audit but does not need to be loaded into the dashboard.

## 9.1 Power Query date fields

```
fact_orders[OrderDate] = Date.From([created_at])
fact_order_items[OrderDate] = Date.From([created_at])
ml_second_purchase_predictions[FirstOrderDate] = Date.From([first_order_at])
```

Set these columns and dim_date[date] to Date type. Close & Apply. Mark dim_date as the Date table using dim_date[date].

# 10. Step 9 - Build the Power BI Semantic Model

Delete the automatically created direct relationship between fact_orders and fact_order_items. Use shared dimensions to filter facts instead. Create these seven one-to-many, single-direction relationships:

| One side | Many side |
|---|---|
| dim_date[date] | fact_order_items[OrderDate] |
| dim_products[id] | fact_order_items[product_id] |
| dim_users[id] | fact_order_items[user_id] |
| dim_date[date] | fact_orders[OrderDate] |
| dim_users[id] | fact_orders[user_id] |
| dim_date[date] | ml_second_purchase_predictions[FirstOrderDate] |
| dim_users[id] | ml_second_purchase_predictions[user_id] |

Keep ml_second_purchase_model_metrics disconnected because it contains report-level model metrics rather than row-level business keys.

# 11. Step 10 - Create DAX Measures

Create the KPI measures used by the report. The following definitions reproduce the implemented logic:

```
Total Revenue = SUM(fact_order_items[revenue])
Total Cost = SUM(fact_order_items[cost])
Total Profit = SUM(fact_order_items[profit])
Profit Margin % = DIVIDE(SUM(fact_order_items[profit]), SUM(fact_order_items[revenue]), 0)
Total Items = COUNTROWS(fact_order_items)

Total Orders = DISTINCTCOUNT(fact_orders[order_id])
Total Customers = DISTINCTCOUNT(fact_orders[user_id])
Average Order Value = DIVIDE(SUM(fact_order_items[revenue]), DISTINCTCOUNT(fact_orders[order_id]), 0)
Revenue per Customer = DIVIDE(SUM(fact_order_items[revenue]), DISTINCTCOUNT(fact_orders[user_id]), 0)
Average Customer Age = AVERAGE(dim_users[age])
```

```
Completed Orders = CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Complete")
Shipped Orders = CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Shipped")
Processing Orders = CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Processing")
Returned Orders = CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Returned")
Cancelled Orders = CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Cancelled")
Return Rate % = DIVIDE(CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Returned"),
DISTINCTCOUNT(fact_orders[order_id]), 0)
Cancellation Rate % = DIVIDE(CALCULATE(DISTINCTCOUNT(fact_orders[order_id]), fact_orders[status] = "Cancelled"),
DISTINCTCOUNT(fact_orders[order_id]), 0)
```

```
Customers Scored = DISTINCTCOUNT(ml_second_purchase_predictions[user_id])
Predicted Repurchase Customers = CALCULATE(DISTINCTCOUNT(ml_second_purchase_predictions[user_id]),
ml_second_purchase_predictions[predicted_repurchase] = 1)
Predicted Repurchase % = DIVIDE(CALCULATE(DISTINCTCOUNT(ml_second_purchase_predictions[user_id]),
ml_second_purchase_predictions[predicted_repurchase] = 1), DISTINCTCOUNT(ml_second_purchase_predictions[user_id]), 0)
Average Repurchase Score = AVERAGE(ml_second_purchase_predictions[repurchase_probability])
High Opportunity Customers = CALCULATE(DISTINCTCOUNT(ml_second_purchase_predictions[user_id]),
ml_second_purchase_predictions[opportunity_segment] = "High Repurchase Probability")
```

Create equivalent Medium and Low Opportunity measures. For model-performance cards, filter ml_second_purchase_model_metrics to dataset="final_test" and return MAX/SELECTEDVALUE for model, roc_auc, pr_auc, precision, recall, f1, accuracy and specificity.

# 12. Step 11 - Build the Six Report Pages

| Page | Minimum visuals required |
|---|---|
| 01 Executive Overview | KPI cards; revenue trend; revenue by traffic source; order-status donut; revenue by country; date and country slicers |
| 02 Sales & Profit | Revenue/profit KPI cards; time trend; category comparison; brand comparison; date slicer |

| | |
|---|---|
| 03 Customer & Marketing | Customer cards; customers by traffic source; customers by country; gender donut; date slicer |
| 04 Product Performance | Top products by revenue/profit; revenue-vs-profit scatter by product ID; product-detail table; category and brand slicers |
| 05 Orders & Returns | Order/return/cancellation cards; status donut; returns vs cancellations over time; country/status stacked chart; date slicer |
| 06 AI - Repurchase Prediction | Scoring cards; opportunity-segment donut; score by traffic source/country; map; customer prediction table; final model metric cards; explanatory text |

For the product scatter plot, use Total Revenue on X, Total Profit on Y, Total Items as size and dim_products[id] as the non-summarized detail key; place product name in tooltips. For the map, use country as Location and High Opportunity Customers as bubble size; latitude/longitude can remain empty because Power BI geocodes country names.

# 13. Step 12 - Validation and Submission

## 13.1 Technical validation

- Refresh Power BI and confirm visuals render without errors.
- Check that date slicers filter the intended fact/prediction tables through dim_date.
- Verify there is no direct fact-to-fact relationship.
- Check the notebook final validation: no duplicate scored first orders, scores remain in [0,1], and Test output row count equals 8,484.
- Use the Databricks final-test metrics as the reference model-evaluation values; Power BI card counts can change with active filter context.

## 13.2 Submission package

| File | Purpose |
|---|---|
| BI_Final_Project.pbix | Interactive Power BI report and semantic model |
| PROJECT.pdf | Six-page static Power BI report export |
| BI_Final_Project.dbc | Databricks notebooks and executed outputs |
| CAT_Project_Analysis_and_Results.pdf | Analytical content and conclusions |
| NvS_Reproducible_Project_Steps.pdf | Rebuild instructions and expected checkpoints |

# 14. Expected Results Checklist

| Checkpoint | Expected value |
|---|---|
| Bronze rows | Users 100,000 \| Orders 125,545 \| Order Items 182,295 \| Products 29,120 |
| Silver rows | Users 100,000 \| Orders 125,545 \| Order Items 182,235 \| Products 29,120 |
| Gold rows | dim_users 100,000 \| dim_products 29,120 \| dim_date 2,821 \| fact_orders 125,545 \| fact_order_items 182,235 |
| Full BI KPIs | Revenue ~10.86M \| Profit ~5.64M \| Margin ~51.9% \| Orders ~126K |
| ML labeled cohort | 56,570 mature first-order customers |
| Selected model | Logistic Regression; threshold 0.50 |
| Final test | ROC-AUC 0.697 \| PR-AUC 0.334 \| Precision 30.6% \| Recall 76.3% \| F1 0.436 |
| Current ML scores | 12,235 rows in Databricks output table |
| Power BI pages | 6 report pages |

If these checkpoints match, the project has been rebuilt successfully from source data through the final analytical and predictive reporting layers.